

SQL Grouping & Filtering

Business Use Case Tutorial

Retail E-Commerce Orders Edition

1. Introduction

This tutorial is designed for data analysts and SQL practitioners who want to master GROUP BY, HAVING, DISTINCT, and related aggregate functions in a real-world business context.

We work with a fictional Retail E-Commerce company that sells products across three categories: Electronics, Home Goods, and Apparel. The business tracks every customer order in a central table called `Sales_Data`. Management needs actionable insights and SQL is the tool that delivers them.

Business Objectives

- Identify top-performing product categories by revenue
- Understand which customer segments are most active
- Filter out low-value transactions for executive reporting
- Spot seasonal trends using date-based grouping
- Combine multiple aggregations for advanced KPI dashboards

Through 5 progressive challenges — from **Easy** to **Hard** — you will write SQL queries that answer real business questions, understand the expected output, and check your work against a full solution key.

2. The Dataset: Sales_Data

The `Sales_Data` table is your primary data source. It contains **15 orders** placed by customers between January and March 2024. Each row represents one order.

Table Schema

Column Name	Data Type	Description
<code>OrderID</code>	INT	Unique identifier for each order
<code>CustomerID</code>	VARCHAR	Unique identifier for each customer
<code>Category</code>	VARCHAR	Product category: Electronics, Home, or Apparel
<code>OrderDate</code>	DATE	Date on which the order was placed (YYYY-MM-DD)
<code>OrderValue</code>	DECIMAL	Total monetary value of the order in USD

Sample Data (15 Rows)

OrderID	CustomerID	Category	OrderDate	OrderValue (\$)
1001	C201	Electronics	2024-01-05	12500
1002	C203	Apparel	2024-01-07	3200
1003	C205	Home	2024-01-10	5800
1004	C201	Electronics	2024-01-15	8900
1005	C207	Apparel	2024-01-18	2100
1006	C210	Home	2024-01-22	4700
1007	C203	Electronics	2024-02-01	15300
1008	C212	Apparel	2024-02-05	1800
1009	C205	Home	2024-02-08	6200
1010	C214	Electronics	2024-02-12	9400
1011	C207	Apparel	2024-02-17	2900
1012	C210	Electronics	2024-02-20	11000
1013	C212	Home	2024-03-01	3500
1014	C201	Apparel	2024-03-05	4100
1015	C214	Home	2024-03-10	7800

Note: CustomerIDs repeat across orders — the same customer may place multiple orders. OrderValues are in USD.

3. The Challenges

Each challenge below presents 3 business questions. Read them carefully, then write your SQL before checking the Expected Output in Section 4 and the Solution Key in Section 5.

LEVEL 1 — EASY Basic GROUP BY with Aggregate Functions

Use `GROUP BY` with functions like `SUM()` and `COUNT()` to aggregate order data by category.

- Q1.** Calculate the total revenue (sum of OrderValue) for each product Category.
- Q2.** Count the total number of orders placed in each Category.
- Q3.** Find the highest (maximum) single OrderValue within each Category.

LEVEL 2 — EASY Using DISTINCT to Identify Unique Segments

Use `DISTINCT` to de-duplicate records and understand the range of values in the dataset.

- Q1.** List all unique product Categories that appear in the Sales_Data table.
- Q2.** List all unique CustomerIDs who have placed at least one order.

Q3. Find all unique combinations of CustomerID and Category (i.e., which customers bought from which categories).

LEVEL 3 — MEDIUM GROUP BY with Multiple Columns

Hint: Group records by more than one column simultaneously to create multi-dimensional summaries.

LEVEL 4 — MEDIUM GROUP BY Combined with HAVING

Hint: The **HAVING** clause filters groups after aggregation — unlike **WHERE**, which filters rows before aggregation.

LEVEL 5 — HARD GROUP BY + HAVING + ORDER BY + Multiple Functions

Hint: Combine **COUNT ()**, **AVG ()**, **SUM ()**, **HAVING**, and **ORDER BY** to answer complex multi-condition business questions.

4. Expected Output

The tables below show the expected query results for each challenge question. Use these to verify your SQL before checking the solution key.

Level 1 — Expected Output

Q1. Total Revenue per Category

Category	Total_Revenue (\$)
Electronics	57,100
Home	28,000
Apparel	14,100

Q2. Total Orders per Category

Category	Total_Orders
Electronics	6
Home	5
Apparel	4

Q3. Max Order Value per Category

Category	Max_OrderValue (\$)
Electronics	15,300
Home	7,800
Apparel	4,100

Level 2 — Expected Output

Q1. Unique Categories

Category
Electronics
Home
Apparel

Q2. Unique CustomerIDs

CustomerID
C201
C203
C205
C207
C210
C212
C214

Q3. Unique Customer + Category Combinations

CustomerID	Category
C201	Electronics
C201	Apparel
C203	Electronics
C203	Apparel
C205	Home
C207	Apparel
C210	Home
C210	Electronics
C212	Apparel
C212	Home
C214	Electronics
C214	Home

Level 3 — Expected Output

Q1. Revenue by CustomerID and Category

CustomerID	Category	Total_Revenue (\$)
C201	Electronics	21,400

CustomerID	Category	Total_Revenue (\$)
C201	Apparel	4,100
C203	Electronics	15,300
C203	Apparel	3,200
C205	Home	12,000
C207	Apparel	5,000
C210	Electronics	11,000
C210	Home	4,700
C212	Apparel	1,800
C212	Home	3,500
C214	Electronics	9,400
C214	Home	7,800

Q2. Order Count by CustomerID and Category

CustomerID	Category	Order_Count
C201	Electronics	2
C201	Apparel	1
C203	Electronics	1
C203	Apparel	1
C205	Home	2
C207	Apparel	2
C210	Electronics	1
C210	Home	1
C212	Apparel	1
C212	Home	1
C214	Electronics	1
C214	Home	1

Q3. Revenue by Category and Month

Category	Order_Month	Total_Revenue (\$)
Electronics	1	21,400
Electronics	2	35,700
Home	1	10,500
Home	2	6,200
Home	3	11,300
Apparel	1	5,300

Category	Order_Month	Total_Revenue (\$)
Apparel	2	4,700
Apparel	3	4,100

Level 4 — Expected Output

Q1. Categories with Revenue > \$20,000

Category	Total_Revenue (\$)
Electronics	57,100
Home	28,000

Q2. Customers with More Than 1 Order

CustomerID	Order_Count
C201	3
C205	2
C207	2
C210	2
C212	2
C214	2

Q3. Customer + Category Combinations Spending > \$10,000

CustomerID	Category	Total_Spent (\$)
C201	Electronics	21,400
C203	Electronics	15,300
C205	Home	12,000
C210	Electronics	11,000

Level 5 — Expected Output

Q1. Category Summary (Avg Order Value > \$5,000, sorted by Revenue)

Category	Total_Orders	Total_Revenue (\$)	Avg_OrderValue (\$)
Electronics	6	57,100	9,516.67
Home	5	28,000	5,600.00

Q2. High-Value Repeat Customers (>1 order, Avg OrderValue > \$6,000)

CustomerID	Order_Count	Avg_OrderValue (\$)
C201	3	8,166.67
C205	2	6,000.00

Q3. Category-Month Summary (Min 2 Orders, Sorted by Month then Revenue)

Category	Month	Orders	Total_Revenue (\$)	Avg_OrderValue (\$)
Electronics	1	3	21,400	7,133.33
Home	1	2	10,500	5,250.00
Apparel	1	2	5,300	2,650.00
Electronics	2	3	35,700	11,900.00
Home	3	2	11,300	5,650.00

5. Solution Key

The following SQL queries produce the expected outputs shown in Section 4. All queries are written in standard SQL (compatible with MySQL, PostgreSQL, and SQL Server with minor adjustments).

Level 1 Solutions — Basic GROUP BY

Q1. Total Revenue per Category

```
SELECT
    Category,
    SUM(OrderValue) AS Total_Revenue
FROM Sales_Data
GROUP BY Category
ORDER BY Total_Revenue DESC;
```

Q2. Total Orders per Category

```
SELECT
    Category,
    COUNT(OrderID) AS Total_Orders
FROM Sales_Data
GROUP BY Category
ORDER BY Total_Orders DESC;
```

Q3. Max Order Value per Category

```
SELECT
    Category,
    MAX(OrderValue) AS Max_OrderValue
FROM Sales_Data
GROUP BY Category
ORDER BY Max_OrderValue DESC;
```

Level 2 Solutions — DISTINCT

Q1. Unique Categories

```
SELECT DISTINCT Category
FROM Sales_Data
ORDER BY Category;
```

Q2. Unique CustomerIDs

```
SELECT DISTINCT CustomerID
FROM Sales_Data
ORDER BY CustomerID;
```


Q3. Unique Customer + Category Combinations

```
SELECT DISTINCT
    CustomerID,
    Category
FROM Sales_Data
ORDER BY CustomerID, Category;
```

Level 3 Solutions — GROUP BY Multiple Columns

Q1. Revenue by CustomerID and Category

```
SELECT
    CustomerID,
    Category,
    SUM(OrderValue) AS Total_Revenue
FROM Sales_Data
GROUP BY CustomerID, Category
ORDER BY CustomerID, Category;
```

Q2. Order Count by CustomerID and Category

```
SELECT
    CustomerID,
    Category,
    COUNT(OrderID) AS Order_Count
FROM Sales_Data
GROUP BY CustomerID, Category
ORDER BY CustomerID, Category;
```

Q3. Revenue by Category and Month

```
-- MySQL / SQL Server syntax:
SELECT
    Category,
    MONTH(OrderDate) AS Order_Month,
    SUM(OrderValue) AS Total_Revenue
FROM Sales_Data
GROUP BY Category, MONTH(OrderDate)
ORDER BY Order_Month, Category;

-- PostgreSQL syntax:
-- GROUP BY Category, EXTRACT(MONTH FROM OrderDate)
```

Level 4 Solutions — GROUP BY with HAVING

Q1. Categories with Revenue > \$20,000

```
SELECT
    Category,
    SUM(OrderValue) AS Total_Revenue
FROM Sales_Data
```

```
GROUP BY Category
HAVING SUM(OrderValue) > 20000
ORDER BY Total_Revenue DESC;
```

Q2. Customers with More Than 1 Order

```
SELECT
    CustomerID,
    COUNT(OrderID) AS Order_Count
FROM Sales_Data
GROUP BY CustomerID
HAVING COUNT(OrderID) > 1
ORDER BY Order_Count DESC;
```

Q3. Customer + Category Spending > \$10,000

```
SELECT
    CustomerID,
    Category,
    SUM(OrderValue) AS Total_Spent
FROM Sales_Data
GROUP BY CustomerID, Category
HAVING SUM(OrderValue) > 10000
ORDER BY Total_Spent DESC;
```

Software Services- Learn To Create

Level 5 Solutions — Advanced Multi-Function Queries

Q1. Category KPI Dashboard (Avg > \$5,000)

```
SELECT
    Category,
    COUNT(OrderID) AS Total_Orders,
    SUM(OrderValue) AS Total_Revenue,
    ROUND(AVG(OrderValue), 2) AS Avg_OrderValue
FROM Sales_Data
GROUP BY Category
HAVING AVG(OrderValue) > 5000
ORDER BY Total_Revenue DESC;
```

Q2. High-Value Repeat Customers

```
SELECT
    CustomerID,
    COUNT(OrderID) AS Order_Count,
    ROUND(AVG(OrderValue), 2) AS Avg_OrderValue
FROM Sales_Data
GROUP BY CustomerID
HAVING COUNT(OrderID) > 1
    AND AVG(OrderValue) > 6000
ORDER BY Avg_OrderValue DESC;
```

Q3. Category-Month Performance (Min 2 Orders)

```
SELECT
    Category,
    MONTH(OrderDate)          AS Order_Month,
    COUNT(OrderID)            AS Total_Orders,
    SUM(OrderValue)           AS Total_Revenue,
    ROUND(AVG(OrderValue), 2) AS Avg_OrderValue
FROM Sales_Data
GROUP BY Category, MONTH(OrderDate)
HAVING COUNT(OrderID) >= 2
ORDER BY Order_Month ASC, Total_Revenue DESC;
```

End of Tutorial — SQL Grouping & Filtering | Retail E-Commerce Use Case
Prepared by Senior Data Analyst & Technical Educator